

```
In [1]: # Behzad Ghabaei
# CS 82A Data Science
# module6_lab2.ipynb
# Textbook Appendix C, Create and Insert
# C.2
# Instructor Seno
# 10/7/2026
```

Step 1: Import sqlite3 and pandas

We establish a connection with the connect() function. Tutorial 1, reaches a real database in the notebook. SQLite stores a whole database in one file. Connect once.

```
con = sqlite3.connect("company.db")
df_employees.to_sql("employees", con, index=False)
```

```
In [2]: import sqlite3
import pandas as pd
# :memory: for testing
con = sqlite3.connect("database.db")
```

Step 2: Create a customers table with four columns.

The code uses the CREATE TABLE statement using SQLite matching types: (INTEGER, TEXT, REAL) The columns should be **id**, **name**, **city**, and a **numeric column**. Using INTEGER, TEXT, REAL.

```
CREATE TABLE movies (
    id INTEGER PRIMARY KEY,
    title TEXT,
    director TEXT,
    year INTEGER,
    length_minutes INTEGER
);
```

([sqlbolt.com](https://sqlbolt.com/lesson/creating_tables) | "SQL Lesson 16: Creating tables" | "An Example" | https://sqlbolt.com/lesson/creating_tables)

```
In [3]: create_table_sql = """
CREATE TABLE IF NOT EXISTS customers (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    city TEXT,
    credit_limit REAL
);
"""
```

One important markdown note is to use the AUTOINCREMENT statement to avoid running into duplicate id problems. This was an issue, due to appending the same data to the output, when different id's were created. Each re start of the python kernel duplicated the data with previous identical data with the new id numbers. To avoid this use AUTOINCREMENT instead of `id INTEGER PRIMARY KEY,`

```
id INTEGER PRIMARY KEY AUTOINCREMENT,
```

AUTOINCREMENT is optional and rarely needed. Without it, SQLite may: reuse old row ids, generate non-sequential IDs, behave differently after table reset. The code above is used

in contrast to using the Step 4 code that also will remedy errors.

```
con.execute("DROP TABLE IF EXISTS customers;")
```

Step 3: Use the INSERT statement with 8 fictional rows.

The numeric value will be called `credit_limit`, because the fictional characters are applying for a pretend credit card. Make sure each row has a unique id

```
In [4]: insert_rows_sql = """
INSERT OR REPLACE INTO customers (id, name, city, credit_limit) VALUES
(1, 'Ali Hussein', 'New York', 5000.00),
(2, 'Bobby Jones', 'Los Angeles', 3500.50),
(3, 'John Brown', 'Chicago', 2500.00),
(4, 'Billy Prince', 'Boston', 7500.00),
(5, 'Eve Kim', 'Austin', 4200.00),
(6, 'Linda James', 'Chicago', 1500.75),
(7, 'Cathy Clark', 'Miami', 6000.00),
(8, 'Hilda Bosche', 'Seattle', 3100.00);
"""
```

Step 4: Using the con.execute() function.

The function `execute()` is used from python. The returned Connection object "con" represents the connection to the on-disk database. Remember to drop the table if it already exists, before recreating it, to ensure you start fresh when re starting the kernel. The examples can be found at [docs.python.org | https://docs.python.org/3/library/sqlite3.html#sqlite3.Connection.execute](https://docs.python.org/3/library/sqlite3.html#sqlite3.Connection.execute)

```
In [5]: #con.execute("DROP TABLE IF EXISTS customers;")
con.execute(create_table_sql)
con.execute(insert_rows_sql)
```

```
Out[5]: <sqlite3.Cursor at 0x1953584e9c0>
```

Step 5: Commit the changes

The INSERT statement implicitly opens a transaction, which needs to be committed before changes are saved in the database

```
In [6]: con.commit()
```

Step 6: Show the database with SELECT *

```
In [7]: df_verification = pd.read_sql_query("SELECT * FROM customers;", con)
df_verification
```

```
Out[7]:
```

	id	name	city	credit_limit
0	1	Ali Hussein	New York	5000.00
1	2	Bobby Jones	Los Angeles	3500.50
2	3	John Brown	Chicago	2500.00
3	4	Billy Prince	Boston	7500.00
4	5	Eve Kim	Austin	4200.00
5	6	Linda James	Chicago	1500.75
6	7	Cathy Clark	Miami	6000.00
7	8	Hilda Bosche	Seattle	3100.00

Step 7: Close the connection.

Any pending transaction is implicitly rolled back. If autocommit is True. Make sure to commit() before closing to avoid losing pending changes.

```
In [8]: con.close()
```

Behzad Ghabaei - End Of Lab